

of a multistatement block inside curly braces is the same as the value returned by the last expression in the block. Therefore, the result of the `formatAbs` method is just the value returned by the call to `msg.format(x, abs(x))`.

Finally, our `main` method is an outer shell that calls into our purely functional core and prints the answer to the console. We'll sometimes call such methods *procedures* (or *impure functions*) rather than functions, to emphasize the fact that they have side effects:

```
def main(args: Array[String]): Unit =
  println(formatAbs(-42))
```

The name `main` is special because when you run a program, Scala will look for a method named `main` with a specific signature. It has to take an `Array` of `Strings` as its argument, and its return type must be `Unit`.

The `args` array will contain the arguments that were given at the command line that ran the program. We're not using them here.

`Unit` serves a similar purpose to `void` in programming languages like C and Java. In Scala, every method has to return some value as long as it doesn't crash or hang. But `main` doesn't return anything meaningful, so there's a special type `Unit` that is the return type of such methods. There's only one value of this type and the literal syntax for it is `()`, a pair of empty parentheses, pronounced "unit" just like the type. Usually a return type of `Unit` is a hint that the method has a side effect.

The body of our `main` method prints to the console the `String` returned by the call to `formatAbs`. Note that the return type of `println` is `Unit`, which happens to be what we need to return from `main`.

## 2.2 Running our program

This section discusses the simplest possible way of running your Scala programs, suitable for short examples. More typically, you'll build and run your Scala code using `sbt`, the build tool for Scala, and/or an IDE like IntelliJ or Eclipse. See the book's source code repo on GitHub (<https://github.com/fpinscala/fpinscala>) for more information on getting set up with `sbt`.

The simplest way we can run this Scala program (`MyModule`) is from the command line, by invoking the Scala compiler directly ourselves. We start by putting the code in a file called `MyModule.scala` or something similar. We can then compile it to Java bytecode using the `scalac` compiler:

```
> scalac MyModule.scala
```

This will generate some files ending with the `.class` suffix. These files contain compiled code that can be run with the Java Virtual Machine (JVM). The code can be executed using the `scala` command-line tool:

```
> scala MyModule
The absolute value of -42 is 42.
```